Axon Flow

A Performant Hierarchical Mind-Mapping Platform
using Client-Side D3-Layouts and Optimistic UI

Akshat Raj (23scse1010833)
Pavan Soni (23scse1012576)

Department of Computer Science & Engineering
Galgotias University

June 2, 2026

Project Overview: What is Axon Flow?

Axon Flow is a full-stack, browser-native mind-mapping application designed for seamless brainstorming and knowledge organization.

- **Infinite Canvas**: Dynamic panning and zooming to view and manage large concept trees.
- **Auto-Layout Modes**: Nodes automatically reposition into Horizontal, Vertical, or Radial hierarchies.
- **Smart Client Architecture**: Relational hierarchy is resolved to physical positions entirely on the user's browser.

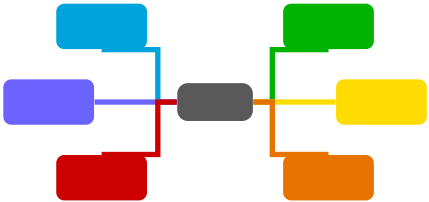
Technology Stack

Frontend: React 18, React Flow (`@xyflow/react`), D3-Hierarchy
Backend: Node.js + Express.js (REST API)
Database: MongoDB Atlas (Mongoose ODM)
Auth: Clerk (JWT-based secure isolation)

Mind Map Visual Topology

Core Topology of Axon Flow:

- **Radial and Orthogonal Layouts**: Clean, automated fanning out of nodes from a central root.
- **Custom Nodes (Leaves)**: Rich nodes holding independent attributes (files, code files, and links).
- **Re-parenting Paths**: Drag-and-drop actions automatically calculate new connection curves.



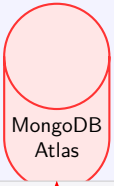
Architectural Weaknesses in Standard Tools

Most existing web-based mind-mapping platforms suffer from three major design and architectural flaws:

1. **Nested-Document Storage**:
   - Storing the entire tree in a single, deeply nested database document.
   - *Consequence*: Changing a single node requires writing back the entire heavy tree, leading to high write costs and sync issues.
2. **Heavy DOM Re-rendering**:
   - Rendering nodes using nested HTML elements (`div` tags) or raw canvasses that repaint completely when nodes move.
   - *Consequence*: Severe performance drops, screen flashing, and lag when maps exceed 100 nodes.
3. **Lack of Contextual Media Support**:
   - Nodes are limited to plain text, lacking ability to bind external files, links, or outlines.

Traditional Database & DOM Bottlenecks

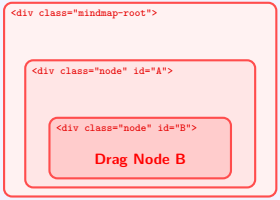
Nested Document Storage (Database)




```
{
  _id: "MAP_001",
  tree: {
    name: "Root Concept",
    children: [
      { name: "Node A",
        children: [
          { name: "Node B" (Edit here) }
        ]
      }
    ]
  }
}
```

Rewrite Cost: 100% of Tree Document
A 1-char rename rewrites the entire tree!

Heavy DOM Re-rendering (Browser)



Cascading Reflow & Layout Paint
Moving B triggers full tree DOM layout cycle!

Axon Flow Core Architecture

1. Flat-Tree Data Model

Every node is stored in MongoDB as an **independent flat document**.

- ✓ Scoped to a map via `mapId`.
- ✓ Linked strictly via a `parentId` reference.
- ✓ Updating a node name or collapsing a branch modifies only **one** database record (`PATCH`).

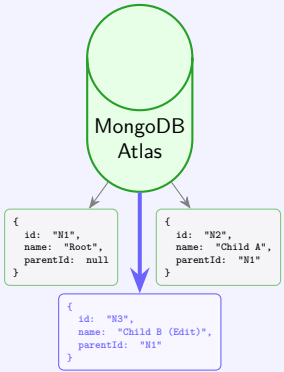
2. Client-Side D3 Geometry

Frontend Layout Engine:

- ✓ The frontend reconstructs the visual tree in memory using D3-Hierarchy.
- ✓ Sibling separation and columns are calculated mathematically in milliseconds based on actual label font widths.

Flat MongoDB Schema & Client-Side D3 Layout

Flat independent Document Schema



✓Single Document Edit (1 Record Updated)
Modifying Node B updates only its own document!

Client-Side Geometry Calculations

1. Fetch Flat JSON Array
[Node1, Node2, Node3]

↓

2. Run D3 stratify()
(Construct N-ary Tree in memory)

↓

3. Run D3 tree() layout
(Calculate X/Y coordinates in 1ms)

↓

4. React Flow DOM Render
(Slide nodes smoothly in browser)

✓Zero Server Latency & 60 FPS Visuals
Layout computed locally on the browser!

Rich Context: Media & Attachment Support

Axon Flow treats mind map nodes as rich portals of information, going far beyond plain text label nodes:

- **File & Code Attachments**:
  - Users can upload PDFs, documents, or developer-centric source code files directly to any node.
  - Stored securely via UUID-prefixed file upload API, with links embedded in the node document.
- **Hyperlink Integrations**:
  - Add multiple external hyperlinks per node, enabling references to tutorials, repositories, or documentations.
- **Granular Inline Data**:
  - Markdown-enabled note lists (`notes[]`) and status badges (e.g., *reading*, *revise*, *completed*, *important*) with custom visual indicators.

Outline Mode: Document-Wise Reading

One major problem with visual maps is **"canvas fatigue"** when trees get too deep. Axon Flow solves this with the **Document-Wise Reading View**:

Linearized Concept Reading:

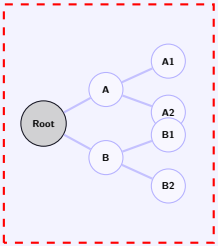
- Toggles the map into a structured, sequential text document.
- Reconstructs the D3 tree into hierarchical headings (e.g., H1, H2, H3) based on depth.
- Renders notes, links, and code snippets inline, reading like a textbook or tutorial.

Why is it better for studying/reading?

1. No panning or searching across the screen.
2. Easier to read long notes and attachments.
3. Combines the power of visual brainstorming with the clarity of linear documentation.

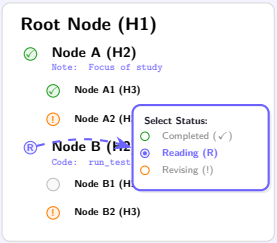
Document-Wise Outline vs. Visual Canvas

Canvas Fatigue (Non-Linear View)



Zoom & Pan Fatigue
Hard to read deep structures!

Linearized Outline (Document View)



Interactive Radial Status Selection
Cycle: Completed (✓) → Reading (R) → Revise (!)

Bulleted-Text Outline Parser

Allows users to convert standard outline text files or lecture notes into interactive visual maps instantly.

Indented Outlines Input:

Listing 1: Bulleted Text Outline

```
Machine Learning
  Supervised
    Regression
    Classification
  Unsupervised
```

Stack-Based Indentation Parser:

- Runs a line-by-line parser tracking indentation depth using a Stack structure.
- Automatically computes correct parent-child references.
- Generates all nodes and pushes them to the DB in a **single bulk API request** (`POST /api/nodes/bulk-create`).

Optimistic UI: Mechanics & Perceived Latency

"Don't make the user wait for the server. Assume success, render instantly."

The Traditional Web Flow

1. User clicks button.
2. Spinner appears. Screen is frozen.
3. Wait for network request (200ms – 1.5s).
4. Server confirms update in database.
5. UI updates. Spinner disappears.

Result: Feels slow, clunky, and native-unfriendly.

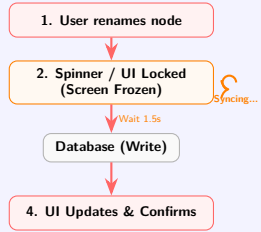
The Optimistic UI Flow

1. User clicks button.
2. **UI updates instantly** based on expected success state.
3. Network request is sent silently in background.
4. If server succeeds: No change, keep editing.
5. If server fails: **Rollback** UI to previous state and notify user.

Result: Feels instantaneous (0ms perceived latency).

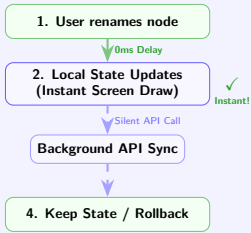
Traditional Request vs. Optimistic UI Timelines

Traditional Flow Timeline



Total Perceived Delay: 1500ms
Laggy, native-unfriendly experience

Optimistic Flow Timeline



Total Perceived Delay: 0ms
Instantaneous, fluid interaction

Optimistic UI: Node Rename Implementation

When a user renames a node bubble on the canvas and presses `Enter`:

Local State Update Logic:

```
// 1. Immediately modify local flat state
setBackendNodes(prevNodes =>
  prevNodes.map(node => {
    if (node.id === targetId) {
      return { ...node, name: newName }; // New obj
    }
    return node; // Unchanged references kept
  })
);

// 2. Fire the API call silently in the background
nodeApi.renameNode(targetId, newName)
  .catch(err => {
    // 3. Rollback logic if backend fails
    revertLocalState(previousNodes);
  });
```

Background Pipeline:

1. User finishes editing inline.
2. State changes immediately.
3. Re-runs React-D3 rendering cycle, adjusting node coordinates.
4. Server updates the MongoDB document in the background.
5. The user experiences zero interruption or delay.

Optimistic UI: Temporary Child Draft Nodes

Adding a new child is complex because we do not have a MongoDB `_id` key until the server responds, yet the canvas must render the new node instantly.

Phase 1: Temporary Draft Node

- Creates a client-only node with a temporary ID: `` `draft-${Date.now()}` ``.
- Injected into the flat list state instantly.
- D3 engine runs, pushes existing nodes away to make room, and displays an input textbox.

Phase 2: Confirmed Swap-Out

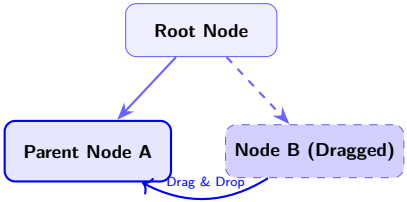
- User hits Enter $\rightarrow$ `POST` request sent.
- Server creates MongoDB record and returns real ObjectId.
- Frontend swaps the draft node object with the real node in state using `map()`.
- Transition is completely invisible to the user.

Optimistic UI: Drag-and-Drop Reparenting

Dragging one node and dropping it onto another changes its parent relationship.

The Interaction Algorithm:

1. **Hit-Test (Dragging)**: A bounding box check (+24px offset) determines if the dragged node is hovering over another node.
2. **Cycle Prevention**: Prevents drops on its own child nodes or itself.
3. **Target Highlight**: Blue pulsing ring indicator.
4. **Optimistic State Update**: On drop release, the node's `parentId` is instantly changed in the local state, triggering a redraw.
5. **Persist & Rollback**: Backend is updated in background. If it fails, the frontend issues a full re-fetch to restore the old state.



DOM Reconciliation: Preventing Screen Flicker

Every time layout math recalculates coordinates for all nodes in the tree, you would expect the browser to reload or flicker. It does not. Here is why:

1. Math vs. Visuals

D3-Hierarchy layout updates coordinates strictly in memory (taking less than **1 millisecond**). It does not touch the HTML elements directly.

2. React Flow Virtual DOM Diffing

React Flow compares node IDs in the old and new arrays. It updates only the CSS inline styles, sliding elements smoothly.

React Flow DOM Translation Output:

```
<!-- BEFORE: X=100, Y=150 -->
<div class="react-flow__node" data-id="NODE_2" style="transform: translate(100px, 150px);">

<!-- AFTER DIFFING: X=50, Y=150 (Only the translate coordinate is modified in DOM) -->
<div class="react-flow__node" data-id="NODE_2" style="transform: translate(50px, 150px);">
```

Coordinate Math & Virtual DOM Diffing

1. D3 Coordinate Calculation

Flat JSON Input (parentId refs):

```
[
  {"id": "N1", "parentId": null},
  {"id": "N2", "parentId": "N1"}
]
```

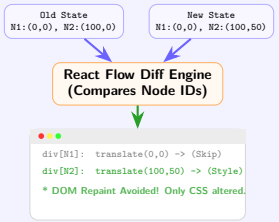
D3 Stratify & Tree Calculation:

- Runs `d3.stratify()` in memory.
- Calculates layout node positions in $<$1ms.

Layout JSON Output (X/Y coordinates):

```
[
  {"id": "N1", "x": 0, "y": 0},
  {"id": "N2", "x": 100, "y": 50}
]
```

2. React Flow DOM Diffing



Render Performance: Complexity & Scalability Limits

Time Complexity Analysis

- **D3 Layout Math**: $\mathbf{O(N)}$
  - Tree reconstruction & layout traversal.
  - In-memory math takes $<$ **1ms** for 500 nodes.
- **Virtual DOM Diffing**: $\mathbf{O(N)}$
  - React compares node IDs using key references.
- **Browser Reflow & Paint**: $\mathbf{O(N \log N)}$
  - Linear reflow (absolute positioning), but SVG edge updates increase paint overhead.

Why it degrades ($>$500 nodes)

- **DOM Node Overload**: 500+ HTML divs + SVG paths exceed browser GPU paint budget.
- **Frame Budget Breached**: Drag updates trigger reflows, exceeding the **16.6ms budget** (FPS drops).

Advanced Optimizations

- **Viewport Virtualization**: Render only nodes visible in the viewport.
- **Canvas/WebGL Rendering**: Bypass DOM by painting directly to Canvas buffer (5,000+ nodes).
- **Web Workers**: Run layout math off the main thread.

Local Generative AI: Node Expansion & Auto-Mapping

Axon Flow integrates a local AI assistant to automate brainstorming and map creation, without relying on cloud APIs.

AI Core Capabilities (Planned)

- **Contextual Node Expansion**: Select any node and trigger the AI to generate 4–6 relevant subtopics as children.
- **Prompt-to-Map (Auto-Map)**: Input a single topic prompt (e.g., *"Deep Learning Basics"*) to spawn a multi-level tree.

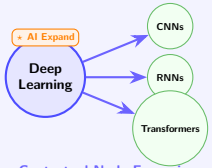
User-Controlled Permissions

Allows enabling/disabling AI tool access:

- **Web Search**: LLM searches DuckDuckGo for live facts.
- **File Reader**: LLM searches DuckDuckGo/local files for attached data.

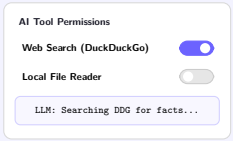
AI Expansion & Tool Permission Controls

AI Map Expansion (Contextual & Auto-Map)



Contextual Node Expansion
Spawns relevant subtopics instantly

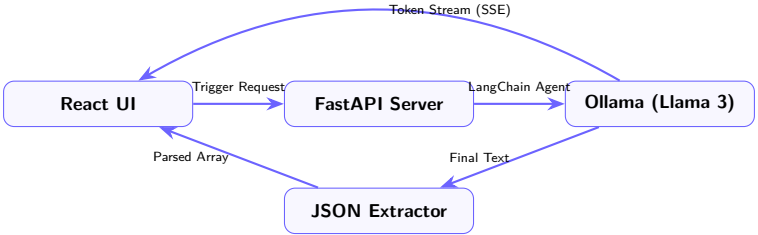
AI Tool Permissions Panel



Granular Tool Controls
User governs local model execution

FastAPI SSE Streaming & JSON Parser

To avoid freezing the UI during LLM inference, Axon Flow streams responses using a FastAPI-based AI engine:



- **SSE Streaming Protocol**: Emits raw text tokens to React's `EventSource` in real-time.
- **JSON Schema Extraction**: Regex-based fallback parser extracts structured array formats from the LLM outputs and maps them into hierarchical parent-child nodes.

Competitive Analysis: Miro/XMind vs. Axon Flow

Dimension	Traditional Tools (Miro / XMind)	Axon Flow
Data Privacy	High risk; data uploaded to cloud LLMs (OpenAI/Anthropic).	**100% private**; runs local model offline via Ollama.
Tech Integrations	Plain text only; no developer attachment capability.	Bind **source code, attachments**, and links to nodes.
Reading Experience	Panning/zooming fatigue on deep visual maps.	Dual view: Switch to linear **Document-Wise Outlines**.
Render Performance	Heavy HTML/canvas redraws causing drag lags.	Optimized flat MongoDB schema + React Virtual DOM translation.

Architectural Pattern: Smart Client, Dumb Server

Axon Flow uses the **"Smart Client, Dumb Server"** pattern to optimize network load and visual smoothness:

The Dumb Backend (Express + MongoDB)	**The Smart Frontend (React + D3)**
• Simply records relationships. • Serves flat lists. • Handles secure authentication and uploads.	• Computes layouts and positions dynamically. • Renders fluid drag-and-drop visuals. • Enables zero-delay Optimistic UI.

Conclusion & Technical Summary

Key Technical Accomplishments

- ✓ **Performance**: Up to 500 nodes rendered smoothly at 60 FPS by running D3 math on client-side and using React Virtual DOM diffs.
- ✓ **Zero Perceived Latency**: Dynamic Optimistic UI updates for creation, renaming, and reparenting actions.
- ✓ **Robust Data Model**: Flat independent node document design in MongoDB scales infinitely better than nested trees.
- ✓ **Enhanced Context**: Attach links, code files, and view outlines in the Document-Wise reader.

Thank You! Questions?